

EVIDENCIA DE PRODUCTO

Evaluación y Certificación de Competencias Laborales - ECCL SENA

****Nombres y apellidos del candidato:** Carlos Daniel Molina Ordoñez**

****Documento:****

****Norma:** 220501096 - Desarrollar solución de software de acuerdo con especificaciones de diseño y marcos de referencia.**

****Fecha de entrega:** 04/09/2026**

DOCUMENTO TÉCNICO DE CÓDIGO FUENTE

Estructura del proyecto

El proyecto está organizado mediante una arquitectura por capas:

```text

Sistema gestion de asociados/

|                 |                                             |
|-----------------|---------------------------------------------|
| — Data/         | Conexión e inicialización del esquema MySQL |
| — Dependencies/ | Integración con el servicio externo de TRM  |
| — Interfaces/   | Contratos de repositorios y servicios       |
| — Models/       | Entidades y modelos del dominio             |
| — Repositories/ | Persistencia y consultas SQL en MySQL       |
| — Services/     | Reglas de negocio y validaciones            |
| — UI/           | Menú e interacción de consola               |
| — Program.cs    | Punto de entrada de la aplicación           |
| — README.md     | Documentación del proyecto                  |

```

Descripción de carpetas y módulos

- ****Data:**** `MySqlDatabase.cs` crea conexiones y genera automáticamente las tablas `Members` y `Movements`.
- ****Models:**** contiene `Member`, `Movement`, `MovementType`, `MemberMovementSummary`, `OperationResult` y `TrmRate`.
- ****Interfaces:**** define los contratos `IMemberRepository`, `IMovementRepository`, `IMemberService`, `IMovementService` e `ITrmService`.
- ****Repositories:**** `MemberRepository` administra asociados y `MovementRepository` administra consignaciones, retiros y reportes mediante SQL parametrizado.
- ****Services:**** `MemberService` aplica las reglas de asociados y `MovementService` controla movimientos, saldos y comisiones.
- ****Dependencies:**** `TrmService` consulta la TRM para convertir saldos de COP a USD.
- ****UI:**** `ConsoleMenu` presenta el menú, recibe entradas y muestra resultados.
- ****Program.cs:**** configura la cultura `es-CO`, crea el menú e inicia la aplicación.

Explicación del flujo del sistema

1. El programa inicia desde `Program.cs`.
2. Se crea `ConsoleMenu` y se inicializa la conexión con MySQL.
3. La aplicación crea las tablas si aún no existen.
4. Se instancian los repositorios MySQL y los servicios de negocio.
5. El usuario selecciona una opción del menú.
6. La interfaz envía los datos al servicio correspondiente.
7. El servicio valida la información y aplica las reglas de negocio.
8. El repositorio ejecuta una consulta parametrizada en MySQL.
9. El resultado se devuelve al servicio y luego se muestra en consola.
10. Los asociados y movimientos permanecen almacenados para futuras ejecuciones.

Fragmentos de código relevantes comentados

Inicialización de la conexión MySQL

```
``csharp
using var connection = CreateConnection();
connection.Open();

// Crea las tablas requeridas si todavía no existen.
using var command = connection.CreateCommand();
command.CommandText = "CREATE TABLE IF NOT EXISTS Members (...)"
command.ExecuteNonQuery();
``
```

Consulta parametrizada

```
``csharp
// Los parámetros separan los datos del usuario de la instrucción SQL.
command.CommandText = ""
    INSERT INTO Members
        (DocumentNumber, FirstName, LastName, Phone, Address, RegistrationDate)
    VALUES
        (@documentNumber, @firstName, @lastName, @phone, @address,
@registrationDate);
    """;
``
```

Inyección de dependencias por constructor

```
``csharp
public MemberService(
    IMemberRepository memberRepository,
    IMovementRepository movementRepository)
{
    _memberRepository = memberRepository;
    _movementRepository = movementRepository;
}
``
```

Cálculo del saldo

```
``csharp
foreach (var movement in movements)
{
    if (movement.Type == MovementType.Deposit)
        balance += movement.Amount;
    else
        balance -= movement.Amount + movement.WithdrawalCommission;
}
``
```

Tecnologías usadas

- C#.
- .NET 10.
- MySQL.
- `MySqlConnection`.
- SQL parametrizado.
- LINQ.
- `HttpClient` y JSON.
- Arquitectura por capas.
- Git y GitHub.
- Aplicación de consola.

INSTRUCTIVO DE USO DE LA SOLUCIÓN DE SOFTWARE

Requisitos del sistema

- Sistema operativo con .NET 10 instalado.
- MySQL Server activo en el puerto `3306`.
- Base de datos `cooperative\_management`.
- Acceso a una terminal.
- Acceso a internet para consultar la TRM.

Pasos de instalación o ejecución

1. Clonar el repositorio:

```
```bash
git clone https://github.com/cxrls7/Sistema-gestion-de-asociados.git
cd Sistema-gestion-de-asociados
```
```

2. Crear la base de datos en MySQL:

```
```sql
CREATE DATABASE IF NOT EXISTS cooperative_management;
```
```

3. Configurar la conexión:

```
```bash
export
MYSQL_CONNECTION_STRING="Server=127.0.0.1;Port=3306;Database=cooperative_management;User ID=root;Password=1234;"
```
```

4. Compilar y ejecutar:

```
```bash
dotnet build
dotnet run
```
```

Las tablas se crean automáticamente al iniciar la aplicación.

Descripción de las funcionalidades

- Registrar asociados.
- Listar asociados con paginación.
- Buscar por documento o nombre.
- Actualizar información de asociados.
- Eliminar asociados sin movimientos ni saldo.
- Registrar consignaciones.
- Registrar retiros con validación de saldo.
- Aplicar comisión a retiros superiores a 1.000.000 COP.
- Consultar saldos en COP y USD.
- Consultar historial de movimientos.
- Generar informes administrativos.
- Consultar movimientos por fecha y los diez movimientos más grandes.

Capturas de pantalla del sistema

****Anexos sugeridos para completar esta sección:****

1. Menú principal de la aplicación en ejecución.
2. Listado de asociados.
3. Registro exitoso de una consignación.

4. Consulta de saldo.
5. Informe administrativo.
6. Tablas `Members` y `Movements` visibles en MySQL Workbench.

> Inserte aquí las capturas tomadas durante la ejecución real de la aplicación y de MySQL Workbench.

SOLUCIÓN DE SOFTWARE

Repositorio en GitHub o GitLab

Repositorio público en GitHub:

<https://github.com/cxrls7/Sistema-gestion-de-asociados>

Código fuente completo

El código fuente completo está disponible en el repositorio público. Incluye:

- Código C# de la aplicación.
- Proyecto `.csproj` y solución `.sln`.
- Modelos, interfaces, servicios y repositorios.
- Integración con MySQL.
- Documentación del proyecto.
- Diagrama del sistema.

Base de datos o scripts SQL

La aplicación crea automáticamente las tablas mediante `Data/MySQLDatabase.cs`. La base inicial se crea con:

```
```sql
CREATE DATABASE IF NOT EXISTS cooperative_management
 CHARACTER SET utf8mb4
 COLLATE utf8mb4_unicode_ci;
```
```

Tablas generadas:

- `Members`: información de los asociados.
- `Movements`: consignaciones y retiros relacionados mediante `MemberId`.

Aplicación ejecutable o API funcionando

La solución es una aplicación de consola ejecutable. Se inicia con:

```
```bash
dotnet run
```
```

El menú principal permite operar el sistema y la persistencia se realiza en MySQL. El repositorio contiene el código fuente completo para compilar y ejecutar la aplicación.

****Firma:**** _____
****Nombres y apellidos del candidato:**** Carlos Daniel Molina Ordoñez
****Documento:**** _____